

📖 GUIA COMPLETO DE ESTUDO – FLASK E EXTENSÕES

Este guia reúne exemplos práticos de Flask, explicando estruturas, métodos, variáveis e funcionalidades, formando uma base sólida para desenvolvimento de aplicações web com Python.

◆ 1. Flask Básico – Estrutura Inicial

```
from flask import Flask
```

```
app = Flask(__name__)
```

Explicação

Flask: framework web em Python

Flask(name): cria a aplicação

__name__ ajuda o Flask a localizar arquivos e recursos do projeto

◆ 2. Rotas e Retorno de Conteúdo

```
@app.route("/")
```

```
def main():  
    return "<h1>Olá Mundo</h1>"
```

Conceitos

@app.route("/"): define a URL

A função retorna o conteúdo que será exibido no navegador

Pode retornar texto, HTML ou templates

◆ 3. Templates HTML com Jinja2

```
from flask import render_template
```

```
@app.route("/")
```

```
def main():  
    return render_template("index.html")
```

Explicação

render_template() carrega arquivos HTML da pasta templates

Permite separar lógica (Python) de visual (HTML)

◆ 4. Formulários, POST e Redirecionamento

```
from flask import Flask, redirect, render_template, request
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```

def main():
    return render_template("teste.html")

@app.route("/numero", methods=["POST"])
def numero():
    n1 = request.form["n1"]
    int(n1)
    return redirect(f"/mostrar/{n1}")

@app.route("/mostrar/<int:n1>")
def mostrar(n1):
    numb = n1
    return render_template("mostrar.html", numb=numb)

```

Explicação

request.form: dicionário com dados do formulário

methods=["POST"]: permite envio de dados

redirect(): redireciona o usuário

/mostrar/int:n1

: rota dinâmica que recebe um número

Variáveis são enviadas ao template (numb)

◆ 5. Banco de Dados com Flask-SQLAlchemy
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy(app)

Explicação

SQLAlchemy é um ORM

Converte classes Python em tabelas

Evita escrever SQL manualmente

◆ 6. Criando Tabelas (Models)
class User(db.Model):
 id = db.Column(db.Integer, primary_key=True)
 username = db.Column(db.String(80))

Conceitos

Cada classe vira uma tabela

primary_key=True gera IDs automáticos

db.Column define colunas

◆ 7. Flask-Login – Autenticação Profissional

```
from flask_login import LoginManager, UserMixin, login_user,  
login_required, current_user
```

Explicação

LoginManager controla autenticação

UserMixin adiciona métodos ao usuário

login_user() autentica o usuário

@login_required protege rotas

current_user representa o usuário logado

◆ 8. Sessão (session) – Controle Simples

```
from flask import session  
session["logado"] = True
```

Conceito

session é um dicionário temporário

Armazena dados enquanto o usuário navega

Muito usado para login simples

◆ 9. Flask-Admin – Painel Administrativo

```
from flask_admin import Admin  
from flask_admin.contrib.sqla import ModelView
```

```
admin = Admin(app, name="Admin")  
admin.add_view(ModelView(User, db.session))
```

Explicação

Cria o painel /admin

Permite CRUD automático

Integra diretamente com o banco

◆ 10. Tema Customizado no Flask-Admin

```
from flask_admin.base import Bootstrap4Theme  
admin_theme = Bootstrap4Theme(swatch="darkly")
```

Explicação

Permite alterar o visual do painel

Usa temas do Bootstrap

◆ 11. Flask-Admin com AdminIndexView (Proteção do Pannel)

```
from flask_admin import AdminIndexView, expose
from flask import session, redirect, url_for
```

```
class MyAdminIndexView(AdminIndexView):
    @expose("/")
    def index(self):
        if not session.get("logado"):
            return redirect(url_for("login"))
        return super().index()
```

Explicação

AdminIndexView controla a página inicial do admin

@expose("/") define a rota interna

session.get("logado") verifica se o usuário está autenticado

super().index() carrega o painel padrão

◆ 12. Fluxo Completo do Pannel Protegido

Usuário tenta acessar /admin

O AdminIndexView verifica a sessão

Se não estiver logado → redireciona para /login

Após autenticação → acesso liberado